

Algorithm 1: Curriculum learning guided by exams and fails

```

Function train(model M, lessons L) :
  for  $\ell \in L$  do
    for  $i = 0, 1, \dots, \ell.\text{max\_epochs}$  do
       $\text{accuracy}, \text{pos}, \text{neg} \leftarrow \text{evaluate}(M, \ell)$ ;           // Take the exam and collect samples.
      if  $\text{accuracy} > \ell.\text{threshold}$  then
        break;                                           // Enter the next lesson if pass the exam.
      for  $j = 0, 1, \dots, K$  do
         $\text{data} \sim \text{balanced sampling from pos and neg}$ ;
        Optimize  $M$  with  $\text{data}$ ;

```

instances with its complexity. Training instances are grouped by their complexity (as *lessons*). For example, in the game of BlocksWorld, we consider the number of blocks in a game instance as its complexity. During the training, we present the training instances to the model from lessons with increasing difficulty. We periodically test models’ performance (as *exams*) on novel instances of the same complexity as the ones in its current lesson. The well-performed model (whose accuracy reaches a certain threshold) will *pass* the exam and advance to a harder lesson (of more complex training instances). The exam-guided curriculum learning exploits the previously gained knowledge to ease the learning of more complex instances. Moreover, the performance on the final exam reaches above a threshold indicates the *graduation* of models.

In our experiments, each lesson contains training instances of the same number of objects. For example, the first lesson in the blocks world contains all possible instances consisting of 2 blocks (in each world). The instances of the second lesson contain 3 blocks in each world. And in the last lesson (totally 11 lessons) there are 12 blocks in each world. We report the range of the curriculum in Table 3 for three RL tasks.

Another essential ingredient for the efficient training of NLMs is to record models’ failure cases. Specifically, we keep track of two sets of training instances: positive and negative (meaning the agent achieves the task or not). For each presented instance of the exam, it is recollected into positive or negative sets depending on whether the agent achieves the task or not. All training samples are sampled from the positive set with probability Ω and from the negative set with probability $1 - \Omega$. This balanced sampling strategy prevents models from getting stuck at sub-optimal solutions. Algorithm 1 illustrates the pseudo-code of the curriculum learning guided by exams and fails.

The evaluation process (“exam”) randomly samples examples from 3 recent lessons. The agent goes through these examples and gets the success rate (the ratio of achieving the task) as its performance, which is used to decide whether the agent passes the exam by comparing to a lesson-depend threshold. As we want a perfect model, the threshold for passing the last lesson (the “final exam”) is 100%. We linearly decrease the threshold by 0.5% for each former lessons, to prevent over-fitting(*e.g.*, the threshold of the first lesson in the blocks world is 95%). After the “exam”, the examples are collected into positive and negative pools according to the outcome (success or not). During the training, we use balanced sampling for choosing training instances from positive and negative pools with probability Ω from positive. The hyper-parameters Ω , the number of epochs, the number of episodes in each training epoch and the number of episodes in one evaluation are shown in Table 3 for three RL tasks.

B IMPLEMENTATION DETAILS AND HYPER-PARAMETERS

This section provides more implementation details for the model and experiments, and summarizes the hyper-parameters used in experiments for our NLM and the baseline algorithm MemNN.

B.1 RESIDUAL CONNECTION.

Analog to the residual link in (He et al., 2016; Huang et al., 2017), we add residual connections to our model. Specifically, for each layer illustrated in Figure 2, the base predicates (inputs) are

concatenated to the conclusive predicates (outputs) group-wisely. That is, input unary predicates are concatenated to the deduced unary predicates while input binary predicates are concatenated to the conclusive binary predicates.

B.2 HYPER-PARAMETERS FOR NLM

Table 4 shows hyper-parameters used by NLM for different tasks. For all MLPs inside NLMs, we use no hidden layer, and the hidden dimension (*i.e.*, the number of intermediate predicates) of each layer is set to 8 across all our experiments. In supervised learning tasks, a model is called “graduated” if its training loss is below a threshold depending on the task (usually $1e-6$). In reinforcement learning tasks, an agent is called “graduated” if it can pass the final exam, *i.e.*, get 100% success rate on the evaluation process of the last lesson.

We note that in the randomly generated cases, the number of maternal great uncle (`IsMGUncle`) relation is relatively small. This makes the learning of this relation hard and results in a graduation ratio of only 20%. If we increase the maximum number of people in training examples to 30, the graduation ratio will grow to 50%.

Table 4: Hyper-parameters for Neural Logic Machines. The definition of depth and breadth are illustrated in Figure 2. “Res.” refers to the use of residual links. “Grad.” refers to the ratio of successful graduation in 10 runs with different random seeds, which partially indicates the difficulty of the task. “Num. Examples/Episodes” means the maximum number of examples/episodes used to train the model in supervised learning and reinforcement learning cases.

	Tasks	Depth	Breath	Res.	Grad.	Num. Examples/Episodes
Family Tree	HasFather	4	3	×	100%	50,000 examples
	HasSister	4	3	×	100%	50,000 examples
	IsGrandparent	4	3	×	100%	100,000 examples
	IsUncle	4	3	×	90%	100,000 examples
	IsMGUncle	4	3	×	20%	200,000 examples
General Graph	AdaJacentToRed	4	3	×	90%	100,000 examples
	4-Connectivity	4	3	×	100%	50,000 examples
	6-Connectivity	8	3	✓	60%	50,000 examples
	1-OutDegree	4	3	×	100%	50,000 examples
	2-OutDegree	5	4	✓	100%	100,000 examples
General Algorithm	Sorting	3	2	✓	100%	1,000 episodes
	Path	5	3	✓	60%	24,000 episodes
	BlocksWorld	7	2	✓	40%	50,000 episodes

B.3 HYPER-PARAMETERS FOR MEMNN

We set the number of iters/episodes used for baseline algorithms to be same as NLM. For the memory networks, each pre-condition in the memory is embedded into a key space and a value space. The dimensions of the spaces are 16 and 32 respectively. The hidden size of the LSTM in MemNN is 64. The number of queries is set to be 4 across all tasks (except that the `Sorting` task uses 1 query only). Empirically, we search for the optimal hyper-parameters but find that they have little effect on the performance.

B.4 DATA GENERATION

We use random generation to generate training and testing data. more details and specific parameters used to generate the data could be found in our open source code.

In family tree tasks, we mimic the process of families growing using a timeline. For each newly created person, we randomly sample the gender and parents (could be none, indicating not included in the family tree) of the person. We also maintain lists of singles of each gender, and randomly pick two from each list to be married (each time when a person was created). We randomly permute the order of people.

In general graph tasks (include `Path`), We adopt the generation method from Graves et al. (2016), which samples m nodes on a unit square, and the out-degree k_i of each node is sampled. Then each node connects to k_i nearest nodes on the unit square. In undirected graph cases, all generated edges are regarded as undirected edges.

In `Sorting`, we randomly generate permutations to be sorted in ascending order.

In `Blocks World`, We maintain a list of placeable objects (the ground included). Each newly created block places on one randomly selected placeable object. Then we randomly shuffle the *id* of the blocks.

B.5 BLOCKS WORLD

In the blocks world environment, to better aid the reinforcement learning process, we train the agent on an auxiliary task, which is to predict the validity or effect of the actions. This task is trained by supervised learning using cross-entropy loss. The overall loss is a summation of cross-entropy loss (with a weight of 0.1) and the REINFORCE loss.

We did not choose the `Move` to be taken directly based on the relational predicates at the last layer of NLM. Instead, we manually concatenate the object representation from the current and the target configuration, which share the same object ID. Then for each pair of objects, their relational representation is constructed by the concatenation of their own object representation. An extra fully-connected layer is applied to the relational representation, followed by a `Softmax` layer over all pairs of objects. We choose an action based on the `Softmax` score.

B.6 ACCURACY DISCUSSION

We cannot directly prove the accuracy of NLM by looking at the induced rules as in traditional ILP systems. Alternatively, we take an empirical way to estimate its accuracy by sampling testing examples. Throughout the experiments section, all accuracy statistics are reported in 1000 random generated data.

To show the confidence of this result, we test a specific trained model of `Blocks World` task with 100,000 samples. We get *no* fail cases in the testing. According to the multiplicative form of Chernoff Bound ⁶, We are 99.7% confident that the accuracy is at least 99.98%.

C NEURAL LOGIC MACHINES (NLM) EXTENSIONS

Reasoning over noisy input: integration with neural perception. Recall that NLM is fully differentiable. Besides taking logic pre-conditions (binary values) as input, the input properties or relations can be derived from other neural architectures (*e.g.*, CNNs). As a preliminary example, we replace the input properties of nodes with images from the MNIST dataset. A convolutional neural network (CNN) is applied to the input extracting multiple features for future reasoning. CNN and NLM can be optimized jointly. This enables reasoning over noisy input.

We modify the `AdjacentToRed` task in general graph reasoning to `AdjacentToNumber0`. In detail, each node has a visual input from the MNIST dataset indicating its number. We say `AdjacentToNumber0( $x$ )` if and only if a node x is adjacent to another node with number 0. We use LeNet LeCun et al. (1998) to extract visual features for recognizing the number of each node. The output of LeNet for each node is a vector of length 10, with sigmoid activation.

⁶[https://en.wikipedia.org/wiki/Chernoff_bound#Multiplicative_form_\(relative_error\)](https://en.wikipedia.org/wiki/Chernoff_bound#Multiplicative_form_(relative_error))